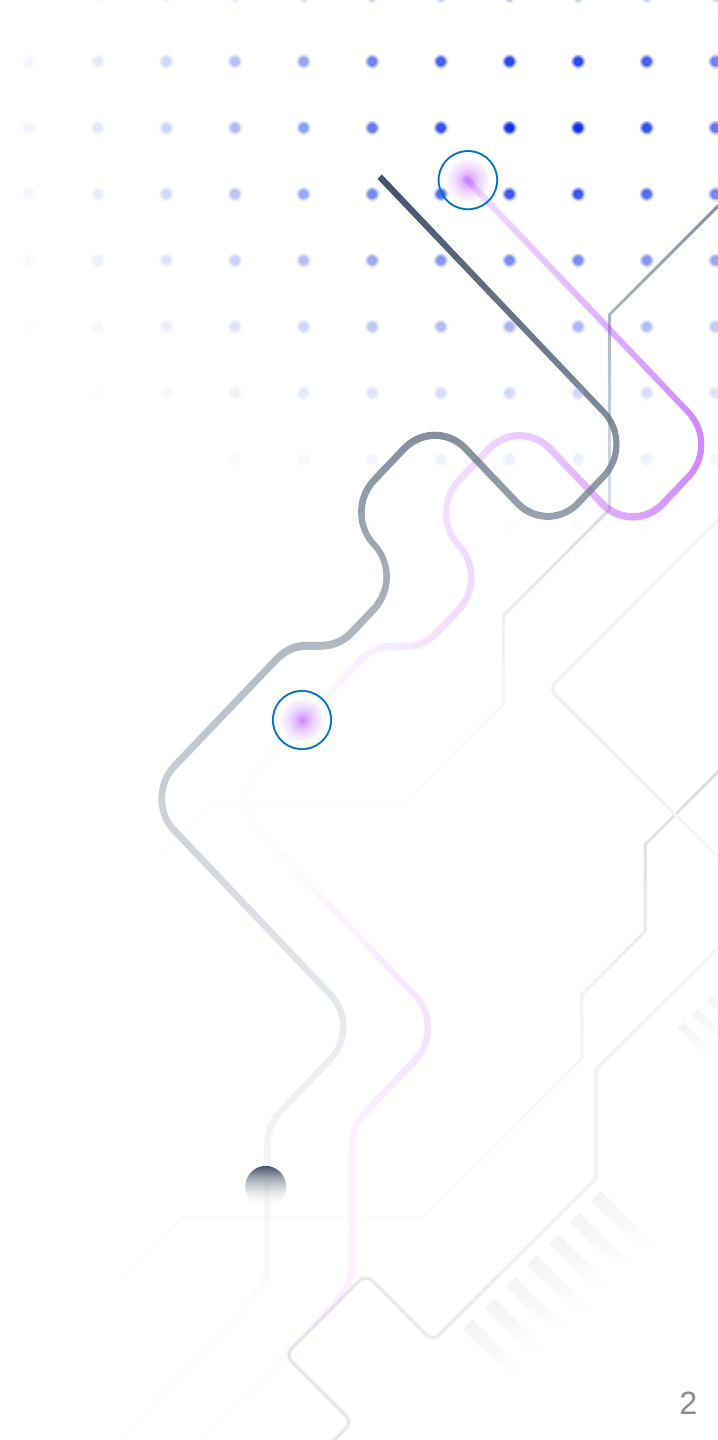


Module: Introduction aux Systèmes Embarqués

Chapitre 02

Syntaxe et Programmation de l'ESP32



Plan du cours

Introduction

Simulateur Wokwi

Carte ESP32 et Kit de développement

Syntaxe de langage de programmation

Plan du cours

Introduction

Simulateur Wokwi

Carte ESP32 et Kit de développement

Syntaxe de langage de programmation

Introduction

- ❑ Ce chapitre a pour objectif de faire une initialisation aux bases de la programmation embarquée sur le microcontrôleur ESP32 en utilisant l'environnement Arduino et le simulateur en ligne Wokwi. Il présente les éléments fondamentaux du langage C/C++ appliqués aux systèmes embarqués, notamment la déclaration des données, l'utilisation des fonctions et la manipulation des entrées/sorties numériques.
- ❑ Objectifs:
 - ❑ Déclarer et utiliser correctement des variables et des constantes,
 - ❑ Comprendre l'impact mémoire des différents types de données,
 - ❑ Structurer un programme embarqué à l'aide de fonctions,
 - ❑ Contrôler des périphériques simples (LEDs) via les GPIO,
 - ❑ Utiliser la communication série pour le débogage.

Plan du cours

Introduction

Simulateur Wokwi

Carte ESP32 et Kit de développement

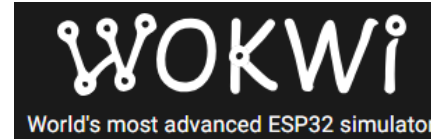
Syntaxe de langage de programmation

Le simulateur wokwi

- ❑ Wokwi est un simulateur électronique en ligne open source gratuit
 - ❑ Multiplateforme
 - ❑ Arduino
 - ❑ ESP32
 - ❑ Raspberry Pi Pico
 - ❑ STM32
 - ❑ Multi-langage
 - ❑ Arduino
 - ❑ MicroPython

Le simulateur wokwi

❑ Connexion Au Simulateur Wokwi



- ❑ Dans un navigateur. saisir l'adresse <https://wokwi.com>
- ❑ Il n'est pas nécessaire de créer un compte pour utiliser le simulateur Wokwi Mais La création d'un compte est recommandée. car cela vous permettra de retrouver “en ligne» vos sketches
- ❑ Pour s'inscrire et créer un compte, cliquer sur le bouton N'inscrire en haut/droite.

Bouton d'inscription



Plan du cours

Introduction

Simulateur Wokwi

Carte ESP32 et Kit de développement

Syntaxe de langage de programmation

Présentation de la carte ESP32



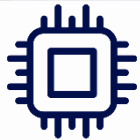
- L'ESP32 est développée par la société Espressif.



- C'est une carte de développement à faible coût dédiée à l'internet des objets (IoT) et les applications embarquées



- C'est un SoC (System on Chip) doté de communications sans fil Wifi et Bluetooth



MCU 32 bits

- Elle est dotée d'un micro-contrôleur 32 bits, est dual core, dispose de 2 processeurs



Fréquence
de 240 MHz



RAM de
512Ko

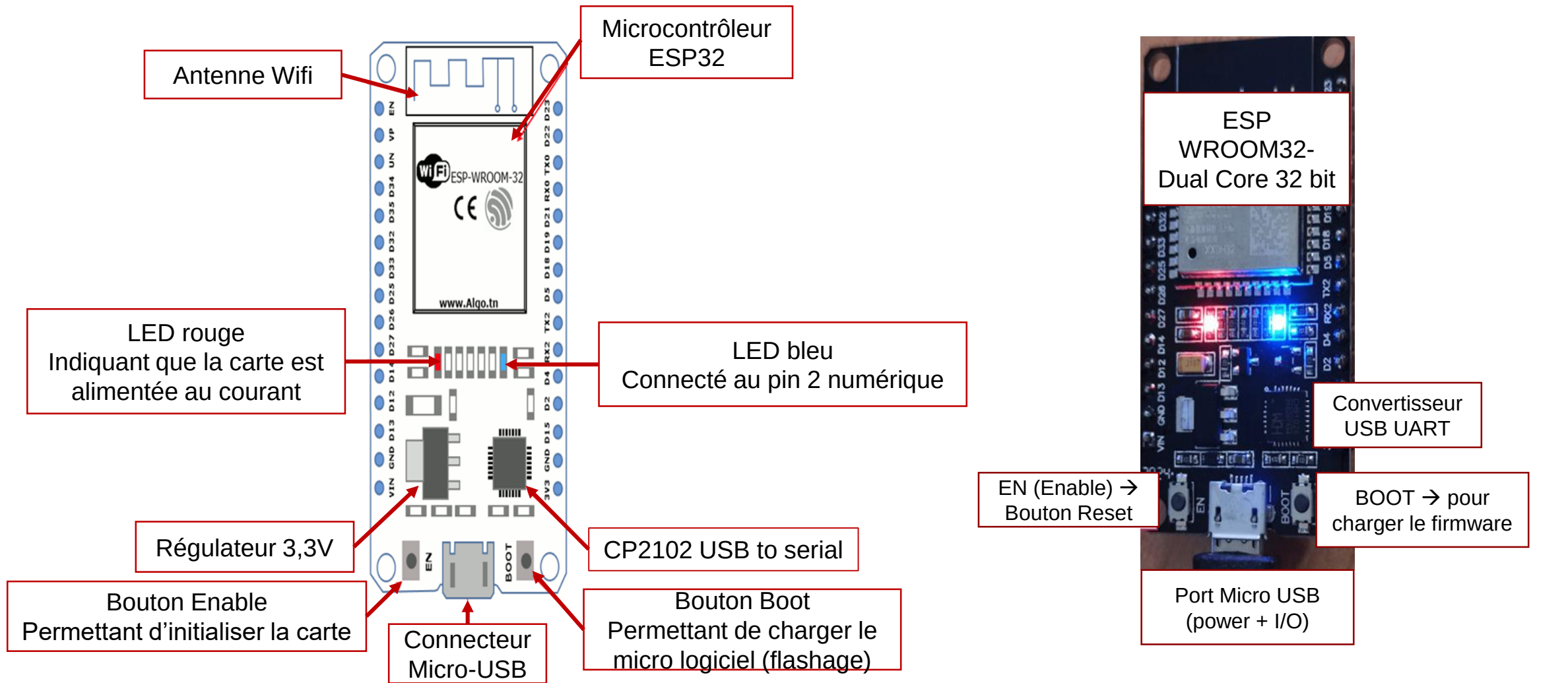


Mémoire flash
de 4MB

- La fréquence d'horloge peut aller jusqu'à 240 MHz et dispose d'une RAM (Random Access Memory) de 512Ko et d'une mémoire flash de 4MB.

1. Carte ESP32
2. Kit de développement

Présentation de la carte ESP32

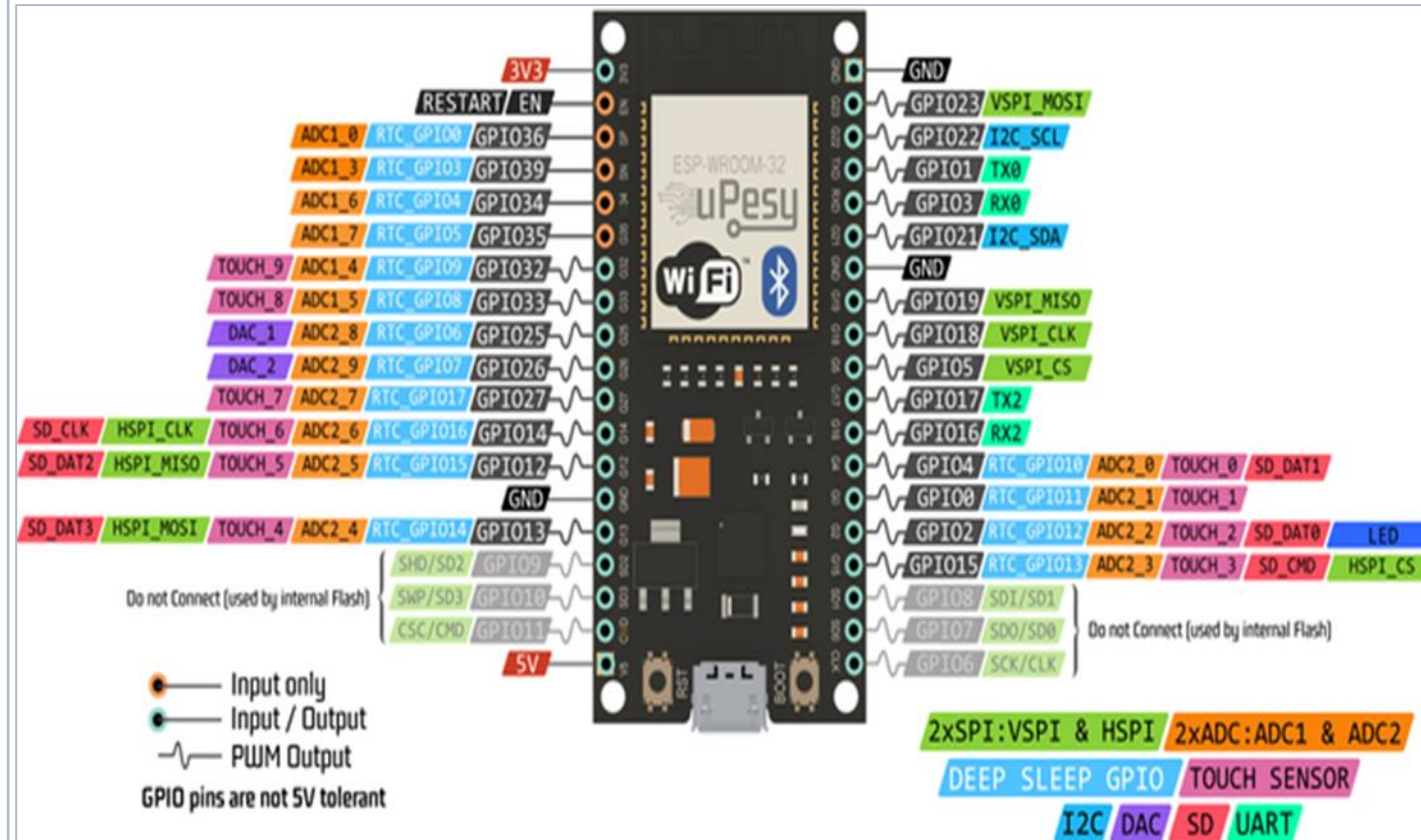


1. Carte ESP32
2. Kit de développement

Présentation de la carte ESP32

Pin ou GPIO:

- GPIO signifie General Purpose Input/Output (Entrée-sortie à usage générale)
- Les ports GPIO sont des ports d'entrées-sorties très utilisés dans le monde des microcontrôleurs

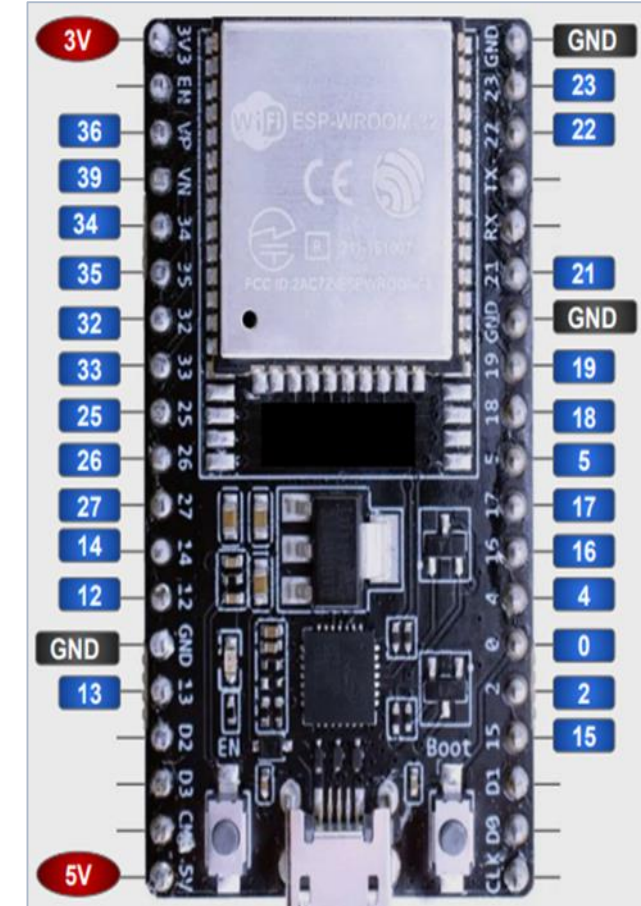


ESP32 Wroom DevKit Full Pinout

Présentation de la carte ESP32

- ❑ **Les pins GPIO6, GPIO7, GPIO8, GPIO9, GPIO10, GPIO11:** Il ne faut surtout pas les utiliser car ils sont reliés à la mémoire flash de l'ESP32: si vous les utilisez l'ESP32 fonctionnera pas.
- ❑ **Les pins GPIO1 (TX) et GPIO3 (RX)** sont utilisés pour communiquer avec l'ordinateur en UART via l'USB. Si vous les utilisez, vous ne pourrez plus uploader de programmes sur la carte ni utiliser le moniteur série via le port USB.
- ❑ **Les pins GPIO36 (VP), GPIO39 (VN), GPIO34, GPIO35** peuvent être utilisés uniquement en entrée.

<https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>



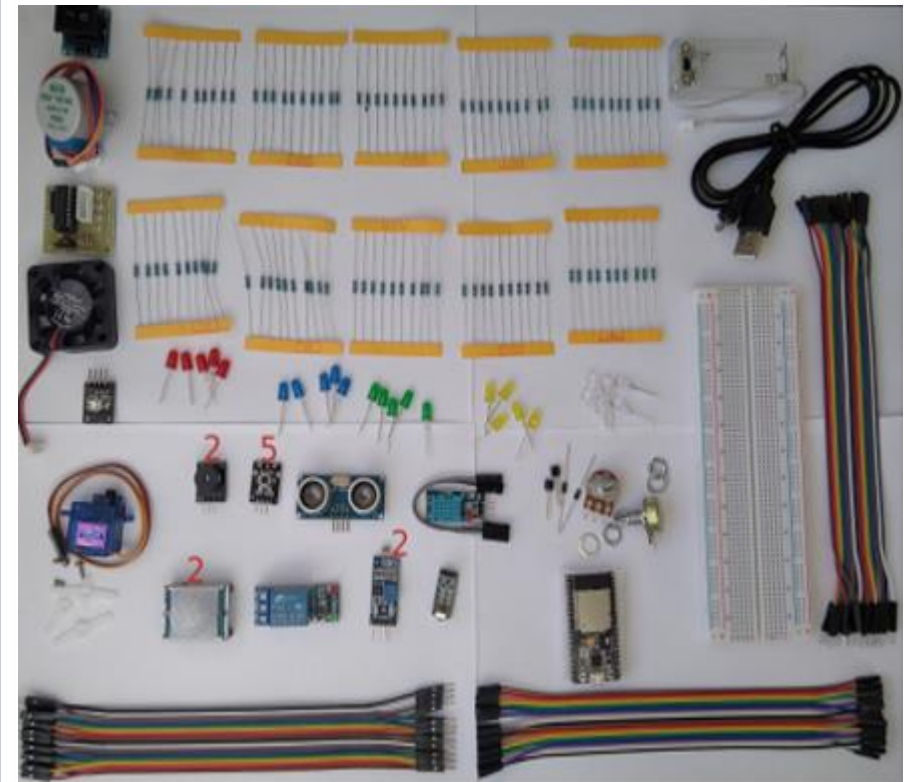
Présentation de la carte ESP32

- ❑ **EN** Enable: broche du régulateur 3.3 V On peut utiliser cette broche connectée à un bouton poussoir et au GND pour redémarrer la carte ESP32.
- ❑ **15 ADC** (Analog-to-Digital Converter) convertisseur analogique-numérique) pour lire les entrées analogiques
- ❑ Les canaux d'entrée ADC ont une résolution de 12 bits. Cela signifie que vous pouvez obtenir des lectures analogiques allant de 0 à 4095, où 0 correspond à 0V et 4095 à 3,3V.
- ❑ 4 interfaces **SPI** (Serial Peripheral Interface): SPI0 et SPI1 (réservés) , HSPI et VSPI (peuvent être utilisés).
- ❑ **GPIO** 34,35,36 et 39 à utiliser comme entrée seulement.
- ❑ **9 capteurs tactiles** capacitifs internes (TOUCH) (GPIO 2,4,15,12,13,14,27,32 et 33). Ceux-ci peuvent détecter des variations dans tout ce qui contient une charge électrique, comme la peau humaine. Ils peuvent ainsi détecter les variations induites lors du contact du GPIO (General-Purpose Input/Output) avec le doigt. Ces broches peuvent remplacer les boutons mécaniques.

- 1. Carte ESP32
- 2. Kit de développement

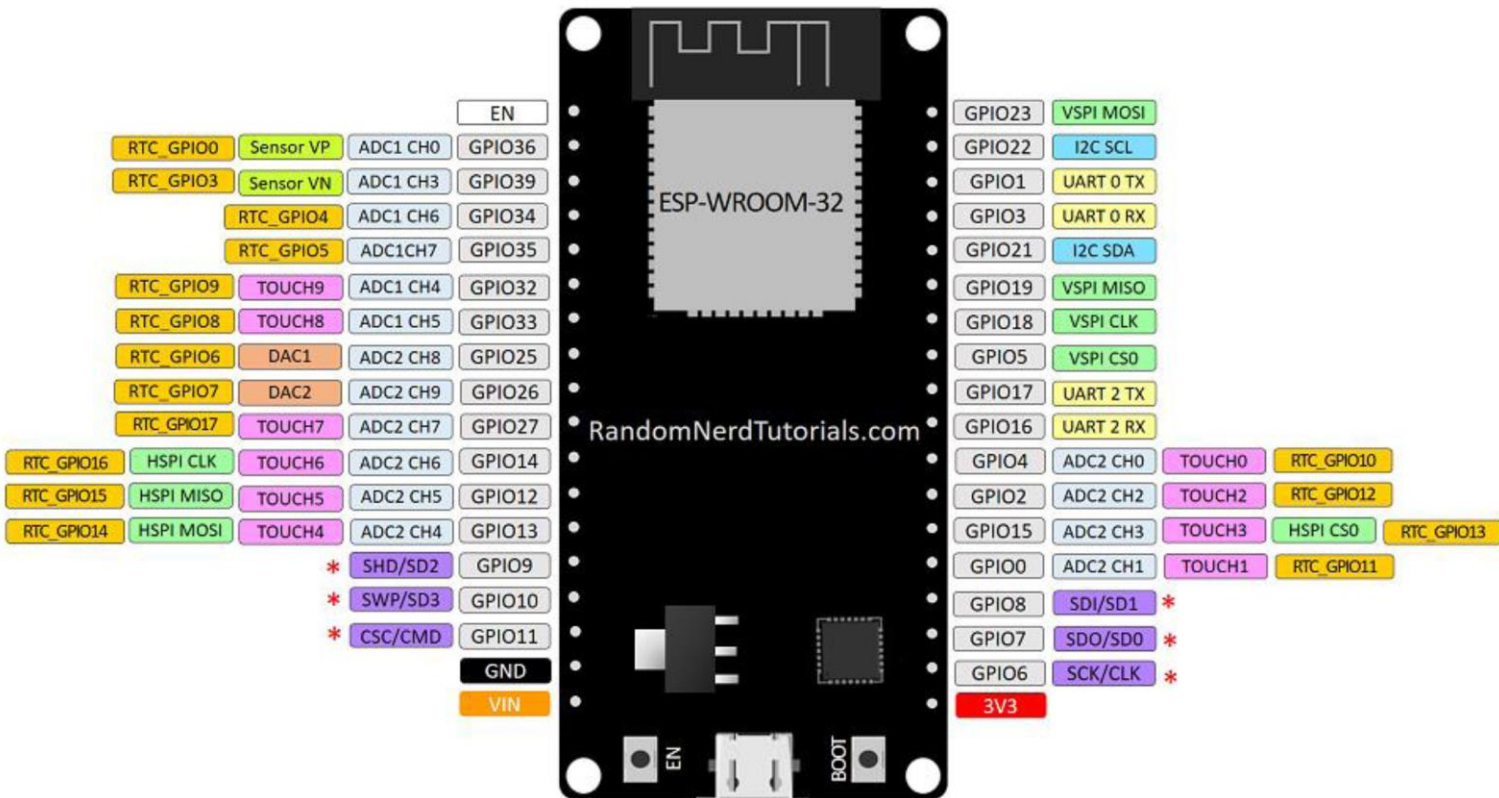
Présentation de la carte ESP32

Découverte du Kit



ESP32 DEVKIT V1 – DOIT

Version avec 36 GPIOs



Les pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3, et SCS/CMD, notamment GPIO6 to GPIO11 sont connectés à SPI flash intégré en ESP-WROOM-32 et ne sont pas recommandés pour autres usages.

[<https://randomnerdtutorials.com/projects-esp32/>]

Plan du cours

Introduction

Simulateur Wokwi

Carte ESP32 et Kit de développement

Syntaxe de langage de programmation

Code initial

- ❑ Le syntaxe d'un langage de programmation est l'ensemble des règles d'écriture liées à ce langage.
- ❑ Nous devons utiliser un code minimal lorsque l'on crée un programme
 - ❑ Ce code permet de diviser le programme que nous allons créer en deux parties principales:

```
// Fonction d'initialisation de la carte
void setup() {
| // Contenu de l'initialisation
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {
| // Contenu de votre programme
}
```

Code initial

```
// Fonction d'initialisation de la carte
void setup() {
    // Contenu de l'initialisation
}
```

- ❑ La fonction setup() est appelée une seule fois lorsque le programme commence. C'est pourquoi c'est dans cette fonction que l'on va écrire le code qui n'a besoin d'être exécutée une seule fois. On appelle cette fonction: fonction d'initialisation. On y retrouvera:
 - ❑ La mise en place des différentes sorties
 - ❑ L'initialisation de votre système
 - ❑ Toutes les programmations que vous ne voulez voir exécutées qu'une fois

Code initial

```
// Fonction principale, elle se répète (s'exécute) à l'infini  
void loop() {  
| // Contenu de votre programme  
}
```

- ❑ C'est donc cette fonction `loop()` où l'on va écrire le contenu du programme. Il faut savoir que cette fonction est appelée en permanence, c'est-à-dire qu'elle est exécutée une fois, puis lorsque son exécution est terminée, on la ré-exécute et encore et encore. On parle de boucle infinie.

Les fonctions de base

Fonction	Role
<code>pinMode(pin, INPUT/OUTPUT);</code>	Configurer un pin soit en entrée soit en sortie
<code>digitalRead(pin)</code>	Lire une valeur numérique à partir d'un pin numérique
<code>digitalWrite(pin, HIGH/LOW);</code>	Met un niveau logique sure un pin numérique soit 1(5v) soit 0 (0v)
<code>Delay(durée);</code>	Faire atteindre (pause) pour le programme à une durée exprimée en Milliseconde

pinMode(pin, mode)

- ❑ Association d'une variable déclarée précédemment à une fonction de type Sortie ou Entrée

```
// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(pin, OUTPUT);
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {
    // Contenu de votre programme
}
```

Delay(ms)

- ❑ Association d'une valeur binaire HIGH ou LOW à la sortie appelée pin.

```
// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(pin, OUTPUT);
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {
    // Passer le pin utilisé à la valeur haute
    digitalWrite(pin, HIGH);
}
```

delay(ms)

❑ Ajout d'un temps de pause (supérieur à celui de l'itération classique) exprimé en millisecondes.

delayMicroseconds(μ s)

❑ Ajout d'un temps de pause (supérieur à celui de l'itération classique) exprimé en microsecondes.

Code sans déclaration

```
// Fonction d'initialisation de la carte  
void setup() {  
    // Association de la connexion digitale à la sortie  
    pinMode(23, OUTPUT); // Définit la broche no: 23 comme une sortie  
}
```

Quel est
l'erreur dans
ce code?

```
// Fonction principale, elle se répète (s'exécute) à l'infini  
void loop() {  
  
    digitalWrite(23, HIGH); // Met la sortie ledPin au Niveau Logique 1 NL1 (diode allumé)  
    delay(3000); // Attendre 3s  
    digitalWrite(ledPin, LOW); // Met la sortie ledPin au Niveau Logique 0 NL0 (diode éteinte)  
    delay(1000); // Attendre 1s  
}
```

❑ Fonction setup()
➤ Contient les instructions
d'initialisation

❑ Fonction loop()
➤ Contient les instructions
du programme



Code sans déclaration

```
// Fonction d'initialisation de la carte
```

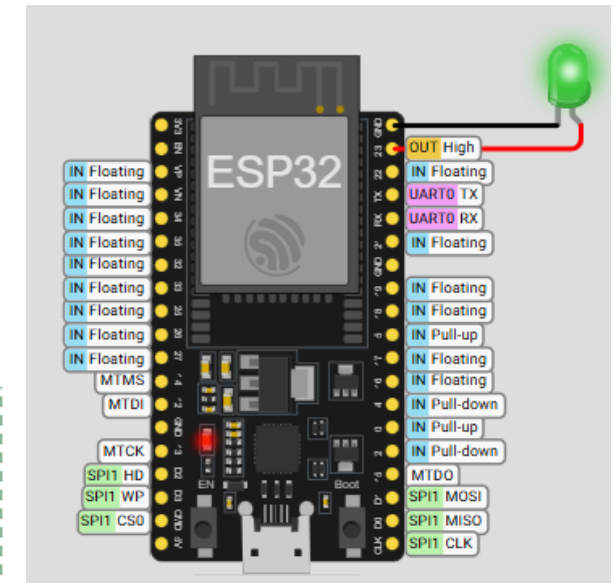
```
void setup() {  
    // Association de la connexion digitale à la sortie  
    pinMode(23, OUTPUT); // Définit la broche no: 23 comme une sortie  
}
```

```
// Fonction principale, elle se répète (s'exécute) à l'infini
```

```
void loop() {  
  
    digitalWrite(23, HIGH); // Met la sortie ledPin au Niveau Logique 1 NL1 (diode allumé)  
    delay(3000); // Attendre 3s  
    digitalWrite(ledPin, LOW); // Met la sortie ledPin au Niveau Logique 0 NL0 (diode éteinte)  
    delay(1000); // Attendre 1s  
}
```

❑ Fonction setup()
➤ Contient les instructions d'initialisation

❑ Fonction loop()
➤ Contient les instructions du programme



23

Les données, variables et constantes

Nom	Description
Boolean	Donnée logique (true ou false) sur 8 bits
Byte	Entier non signé sur 8 bits
int	Entier signé sur 16 bits
Unsigned int	Entier non signé sur 16 bits
Long	Entier signé sur 32 bits
Unsigned long	Entier non signé sur 32 bits
Float	Nombre à virgule flottant sur 32 bits
Char	Caractère sur 8 bits. Seuls les caractères ayant pour code ASCII une valeur 0 à 127 sont définis

Les données, variables et constantes

- ❑ Une constante peut être définie au moyen de `const` ou de `#define`:

```
#define N 2 // Toute variable N rencontrée dans le programme sera remplacée par la valeur 2
const byte N=2 // N est une constante de type "byte" et de valeur 2
```

- ❑ Un certain nombre de noms de constantes sont prédéfinis

Nom	Description
true	Donnée binaire égale à 1 ou donnée numérique différente de 0
false	Donnée binaire ou numérique égale à 0
HIGH	Génération d'un niveau de tension haut sur une des sorties numériques
LOW	Génération d'un niveau de tension bas sur une des sorties numériques
INPUT	Configuration d'une broche numérique en entrée
OUTPUT	Configuration d'une broche numérique en sortie

Les données, variables et constantes

❑ Double & String

Nom	Description
double	Ce type offre une précision deux fois supérieure à celle du float. Dans le cas du microcontrôleur Arduino, les deux types sont
String	Ce n'est plus vraiment un type, mais un Objet, au sens de la POO. Pour faire simple dans un premier temps, considérons qu'il s'agit d'une chaîne de caractère

#define

❑ Description : est utilisé pour créer des constantes qui remplacent un nom par une valeur spécifiée dans le code.

Cela se fait avant la compilation du programme.

❑ Mémoire réservée : 0 octet (aucune mémoire n'est allouée dans la RAM)

➤ Explication matérielle : Comme #define remplace le nom par la valeur dans le code source, il n'y a pas d'utilisation de mémoire RAM.

```
#define LED_PIN 23 // Définit LED_PIN comme 23

// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(LED_PIN, OUTPUT); // Configure la broche LED_PIN comme sortie
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {

    digitalWrite(LED_PIN, HIGH); // diode allumée
    delay(1000); // Attendre 1s
    digitalWrite(LED_PIN, LOW); // diode éteinte
    delay(1000); // Attendre 1s
}
```

Const int

❑ Description : `const int` est utilisé pour définir une constante

entière qui ne peut pas être modifiée après son initialisation. Cela permet d'assurer que la valeur reste fixe

tout au long de l'exécution du programme.

❑ Mémoire réservée : 2 octets (pour un entier).

```
const int ledPin = 13; // Définit ledPin comme 13

// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(ledPin, OUTPUT); // Configure la broche ledPin comme sortie
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {

    digitalWrite(ledPin, HIGH); // diode allumée
    delay(1000); // Attendre 1s
    digitalWrite(ledPin, LOW); // diode éteinte
    delay(1000); // Attendre 1s
}
```

int

❑ Description : `int` est utilisé pour déclarer des variables qui peuvent contenir des nombres entiers. Contrairement aux constantes, la valeur d'un `int` peut être modifiée à tout moment pendant l'exécution du programme.

❑ Mémoire réservée : 2 octets (pour un entier)

```
int ledPin = 13; // Définit ledPin comme 13

// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(ledPin, OUTPUT); // Configure la broche ledPin comme sortie
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {
    ledPin = 22; // Modifie la valeur de ledPin
    pinMode(ledPin, OUTPUT); // Reconfigure la nouvelle broche
    digitalWrite(ledPin, HIGH); // diode allumée
    delay(1000); // Attendre 1s
    digitalWrite(ledPin, LOW); // diode éteinte
    delay(1000); // Attendre 1s
    ledPin = 13; // Remet la valeur d'origine
}
```


long

- ❑ Description: long est utilisé pour déclarer des variables qui peuvent contenir des nombres entiers plus grands que ceux autorisés par int. Contrairement aux constantes, la valeur d'un long peut être modifiée à tout moment pendant l'exécution du programme.
- ❑ Mémoire réservée: 4 octets (pour un entier long).
- ❑ Explication matérielle: Les variables de type long sont stockées en RAM et peuvent être modifiées à chaque itération du programme.

```
long ledPin = 13; // Définit ledPin comme 13

// Fonction d'initialisation de la carte
void setup() {
    // Association de la connexion digitale à la sortie
    pinMode(ledPin, OUTPUT); // Configure la broche ledPin comme sortie
}

// Fonction principale, elle se répète (s'exécute) à l'infini
void loop() {
    ledPin = 22; // Modifie la valeur de ledPin
    pinMode(ledPin, OUTPUT); // Reconfigure la nouvelle broche
    digitalWrite(ledPin, HIGH); // diode allumée
    delay(1000); // Attendre 1s
    digitalWrite(ledPin, LOW); // diode éteinte
    delay(1000); // Attendre 1s
    ledPin = 13; // Remet la valeur d'origine
}
```

static

❑ Description: static : Variable locale conservant sa valeur entre les appels. Utile pour les boucles.

```
void setup() {  
    // Association de la connexion digitale à la sortie  
    pinMode(23, OUTPUT); // Configure la broche 23 comme sortie  
}  
  
void loop() {  
    static int ledPin = 23; // Déclaration d'une variable locale static  
  
    digitalWrite(ledPin, HIGH); // diode allumée  
    delay(1000); // Attendre 1s  
    digitalWrite(ledPin, LOW); // diode éteinte  
    delay(1000); // Attendre 1s  
}
```

byte

❑ Avec ce programme, pour faire clignoter le LED avec une autre vitesse ou un autre port, il suffit de changer à un seul endroit les valeurs affectées aux constantes `dureePause` et `brocheLED`. Ici le type de `brocheLED` est `const byte`: le type `byte` est suffisant pour représenter tous les broches de la carte car leur nombre ne dépasse pas 256, et constante puisqu'elle ne changera pas de valeur au cours du programme. De même, le type `unsigned` de `dureePause` est largement suffisant pour notre application

```
/* Clignoter une LED */
const byte brocheLED = 13;
const unsigned dureePause = 1000; // la valeur de dureePause est fixée

void setup() {
  pinMode(brocheLED, OUTPUT); // Port 13 = builtin LED (La LED L)
}

void loop() {

  digitalWrite(brocheLED, HIGH);
  delay(dureePause);
  digitalWrite(brocheLED, LOW);
  delay(dureePause);
}
```

1. Code initial
2. Fonctions de base
3. Code sans déclaration

4. Données, variables et constantes
5. Usage et exercices

Utilisation des fonctions C

```
const int ledPin = 13; // Déclaration du pin pour la LED

// Fonction pour allumer et éteindre la LED
void led_R()
{
    digitalWrite(ledPin, HIGH); // Allume la LED
    delay(1000);                // Attendre 1 seconde
    digitalWrite(ledPin, LOW);  // Eteindre la LED
    delay(1000);                // Attendre 1 seconde
}

void setup() {
    pinMode(ledPin, OUTPUT); // Configure le pin 23 comme sortie
}

void loop() {

    led_R(); // Appelle la fonction pour controler la LED
}
```



Utilisation des fonctions C

```
const int ledpin = 23; // Déclaration de la
broche de la LED
```

```
void setup() {
  pinMode(ledpin, OUTPUT); //
  Configuration de la broche en sortie
}
```

```
void led_R() {
  digitalWrite(ledpin, HIGH);
  delay(1000); // Allume la LED pendant 1 seconde
  digitalWrite(ledpin, LOW);
  delay(1000); // Éteint la LED pendant 1 seconde
}

void loop() {
  led_R(); // Appel de la fonction pour faire clignoter la LED
}
```

Contrôle d'une LED avec fonction de retour

```
int ledPin = 13; // Déclaration du pin pour la LED
int duree = 1000; // Durée par défaut de 1 seconde

void setup() {
  pinMode(ledPin, OUTPUT); // Configure le pin 23 comme sortie
}

// Fonction pour allumer et éteindre la LED
void led_R(int etat, int duree)
{
  // Etat 1: 1 pour HIGH (Allumé), 0 pour LOW (éteint)
  digitalWrite(ledPin, etat); // Allume la LED
  delay(duree);               // Maintient cet état pendant la durée spécifiée
}

void loop() {
  // Allume la LED pendant la durée spécifiée
  led_R(HIGH, duree); // HIGH signifie que la LED est allumée
  // Eteint la LED pendant la durée spécifiée
  led_R(LOW, duree); // LOW signifie que la LED est éteinte
}
```

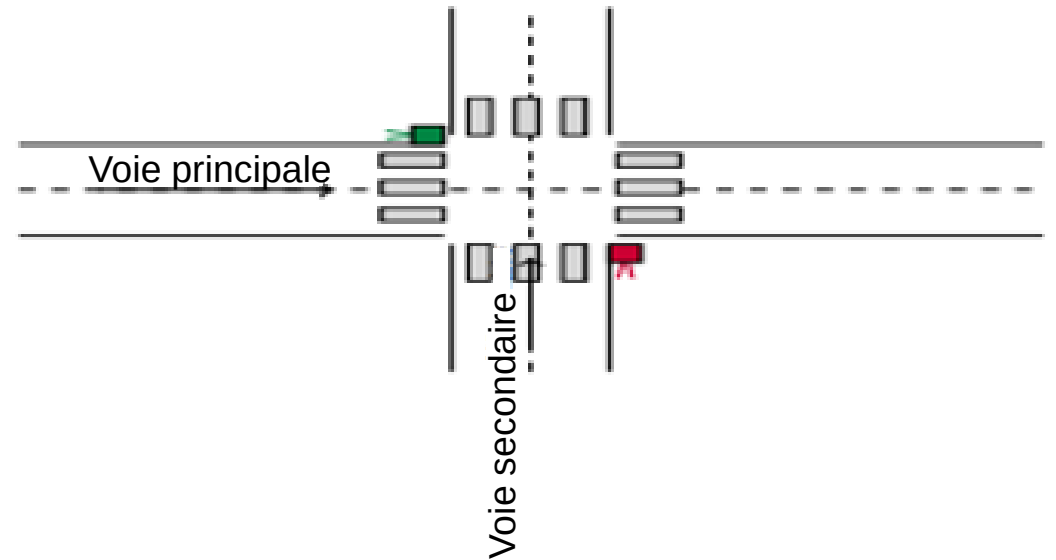
Exercice: Programmation de feux rouges: Problème 1

❑ Objectif

- ❑ Programmer un ensemble urbain avec une modélisation des feux de croisement sur carte Arduino.
- ❑ On considère deux voies de circulation a sens unique avec un croisement comportant donc deux feux tricolores.

❑ Problème 1:

- ❑ En journée, la programmation souhaitée est la suivante:
 - ❑ Le feu principal passe au vert durant 5 secondes;
 - ❑ Il passe alors au orange pendant 2 secondes;
 - ❑ Les deux feux restent au rouge durant 1 seconde;
 - ❑ Le feu secondaire passe au vert durant 5 secondes;
 - ❑ Il passe alors au orange pendant 2 secondes;
 - ❑ Les deux feux restent au rouge durant 1 seconde;
- ❑ Réaliser une programmation et un montage permettant de respecter cette coordination de feux de croisement.



Exercice: Programmation de feux rouges: Problème 1

❑ Test du programme

```
// Définition des broches
const int led_rouge_feux_1 = 2;
const int led_jaune_feux_1 = 3;
const int led_verte_feux_1 = 4;
const int led_rouge_feux_2 = 8;
const int led_jaune_feux_2 = 9;
const int led_verte_feux_2 = 10;
```

```
void setup() {
    // Initialisation en sortie de toutes les broches
    pinMode(led_rouge_feux_1, OUTPUT);
    pinMode(led_jaune_feux_1, OUTPUT);
    pinMode(led_verte_feux_1, OUTPUT);
    pinMode(led_rouge_feux_2, OUTPUT);
    pinMode(led_jaune_feux_2, OUTPUT);
    pinMode(led_verte_feux_2, OUTPUT);

    // On initialise toutes les LEDs éteintes au début du programme
    // (Sauf les deux feux rouges)
    digitalWrite(led_rouge_feux_1, HIGH);
    digitalWrite(led_jaune_feux_1, LOW);
    digitalWrite(led_verte_feux_1, LOW);
    digitalWrite(led_rouge_feux_2, HIGH);
    digitalWrite(led_jaune_feux_2, LOW);
    digitalWrite(led_verte_feux_2, LOW);
}
```


Exercice: Programmation de feux rouges: Problème 1

```
void loop() {  
    delay(1000); // this speeds up the simulation  
    // Première séquence  
    // 1 Passe au vert  
    digitalWrite(led_verte_feux_1, HIGH);  
    digitalWrite(led_rouge_feux_1, LOW);  
    // 2 Reste au rouge  
    delay(5000);  
    // Deuxième séquence  
    // 1 Passe au orange  
    digitalWrite(led_jaune_feux_1, HIGH);  
    digitalWrite(led_verte_feux_1, LOW);  
    // 2 reste rouge  
    // Troisième séquence  
    // 1 Passe au rouge  
    digitalWrite(led_rouge_feux_1, HIGH);  
    digitalWrite(led_jaune_feux_1, LOW);  
    // 2 reste rouge  
    delay(1000);  
}
```

```
// Quatrième séquence  
// 2 Passe au vert  
digitalWrite(led_verte_feux_2, HIGH);  
digitalWrite(led_rouge_feux_2, LOW);  
// 1 Reste au rouge  
delay(5000);  
// Cinquième séquence  
// 2 Passe au orange  
digitalWrite(led_jaune_feux_2, HIGH);  
digitalWrite(led_verte_feux_2, LOW);  
// 1 reste rouge  
// Sixième séquence  
// 2 Passe au rouge  
digitalWrite(led_rouge_feux_2, HIGH);  
digitalWrite(led_jaune_feux_2, LOW);  
// 1 reste rouge  
}
```

Fonction digitalWrite()

❑ Description: Lecture d'une entrée déclarée et renvoie d'une valeur binaire HIGH ou LOW

```
int pin = 7;
int val = 0;

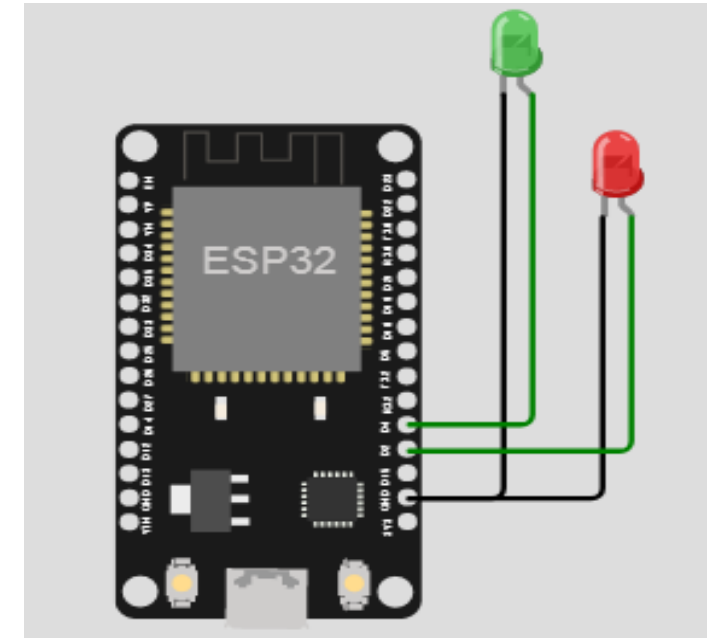
void setup() {
    // Association de la connexion digitale à une entrée
    pinMode(pin, INPUT);
}

void loop() {
    // Lecture de l'entrée définie
    val = digitalWrite(pin);
}
```

Exercice: Contrôle de deux LEDs avec digitalWrite()

❑ Ecrire un programme en langage C pour contrôler deux LEDs connectées aux pins 2 et 4 de votre microcontrôleur.

1. Déclarez et initialisez les variables pin2 et pin 4 avec les valeurs respectives 2 et 4.
2. Configurez ces pins en tant que sorties dans la fonction setup().
3. Dans la fonction loop(), allumez la LED connectée au pin 2, lisez son état, et affectez cet état à la LED sur le pin 4
4. Attendez une seconde, puis éteignez la LED du pin 2 et mettez à jour l'état sur le pin 4 avant de répéter le cycle.



1. Code initial
2. Fonctions de base
3. Code sans déclaration

4. Données, variables et constantes
5. Usage et exercices

Exercice: Contrôle de deux LEDs avec digitalWrite()

```
const int LEDR = 2; // Broche de la LED rouge
const int LEDV = 4; // Broche de la LED verte

int val = 0; // Variable pour stocker l'état de LEDR

void setup() {
  pinMode(LEDR, OUTPUT);
  pinMode(LEDV, OUTPUT);

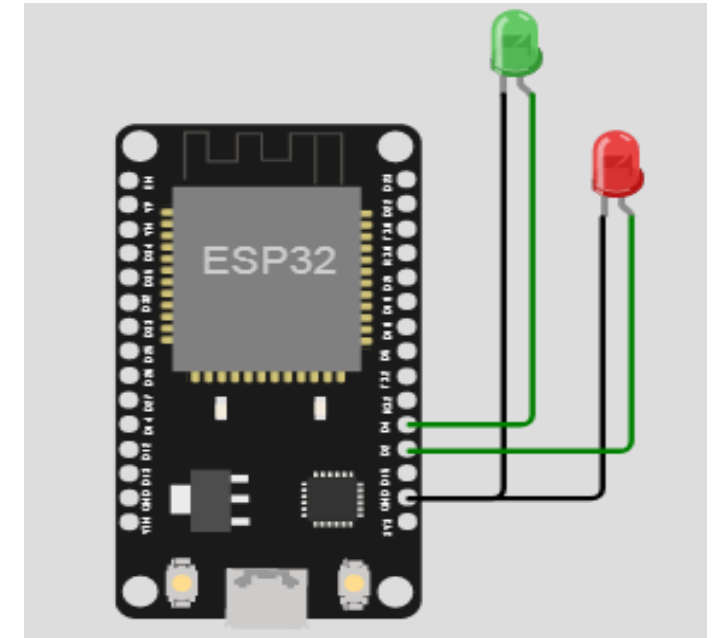
  // Allumer la LED rouge une seule fois
  digitalWrite(LEDR, HIGH);

  // Attendre 500 ms avant d'allumer la LED verte
  delay(500);

  // Lire l'état de LEDR et l'affecter à val
  val = digitalRead(LEDR);

  // Copier l'état de LEDR sur LEDV après le délai
  digitalWrite(LEDV, val); // LEDV prend le même état que LEDR
}

void loop() {
  // Vide pour éviter de répéter l'action
}
```



Affichage des sorties

- ❑ Afin de pouvoir consulter les sorties, il faut paramétrer la communication:

```
void setup()
{
  Serial.begin(9600);
}
void loop()
{
}
```

- ❑ Cela fixe le débit de communication en nombre de caractères par secondes (unité: baud) pour la communication série;
- ❑ Pour communiquer avec l'ordinateur, il faut utiliser un de ces débits: 300, 1200, 2400, 4800 ... 115200.

Plus le débit est rapide, plus la communication est rapide.

Affichage des sorties

- ❑ Une fois que la communication est paramétrée, il est possible d'afficher les sorties analogiques ou numériques.

```
void setup()
{
  Serial.begin(9600);
}
void loop()
{
  Serial.print(val);
}
```

- ❑ Pour afficher ces sorties, il suffit alors d'activer le moniteur et de le régler à la bonne fréquence

La fonction `Serial.println(val)` inclut un retour à la ligne

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  Serial.println("Hello, bonjour");
}

void loop() {
  // put your main code here, to run
  repeatedly:
  delay(10); // this speeds up the simulation
}
```

1. Code initial
2. Fonctions de base
3. Code sans déclaration

4. Données, variables et constantes
5. Usage et exercices

Affichage des sorties

```
void setup() {  
    Serial.begin(9600); // Initialise la communication série à une vitesse de 9600 bauds  
}  
  
void loop() {  
    float temperature = 24.2345;  
  
    Serial.print(temperature, 1); // Affiche la valeur de la température avec 1 chiffre après la virgule  
    Serial.println(); // Effectue un saut de ligne  
  
    Serial.print(temperature, 2); // Affiche la valeur de la température avec 2 chiffres après la virgule  
    Serial.println(); // Effectue un saut de ligne  
  
    Serial.print(temperature, 3); // Affiche la valeur de la température avec 3 chiffres après la virgule  
    Serial.println(); // Effectue un saut de ligne  
    delay(1000); // Attend 1 seconde  
}
```

```
void setup() {  
    Serial.begin(9600); // Initialise la communication  
}  
  
void loop() {  
    float temperature = 24.2345;  
  
    Serial.print("Température : "); // Affiche le mot  
    Serial.print(temperature, 1); // Affiche la valeur  
    Serial.println(); // Effectue un saut de ligne  
  
    Serial.print("Température : "); // Affiche le mot  
    Serial.print(temperature, 2); // Affiche la valeur  
    Serial.println(); // Effectue un saut de ligne  
  
    Serial.print("Température : "); // Affiche le mot  
    Serial.print(temperature, 3); // Affiche la valeur  
    Serial.println(); // Effectue un saut de ligne  
  
    delay(1000); // Attend 1 seconde  
}
```